02613 Python and High Performance Computing

Answers for Week 7

For these exercises, we will use weather data from public DMI (Danmarks Metereologiske Institut) from January 2023. You can read more about DMI's free data (in Danish) here. For information about the data records we will be working with (in English), see the official documentation. We have prepared the relevant data in a CSV file you can find at `/dtu/projects/02613_2025/data/dmi/2023_01.csv.zip`.

Each row of the CSV file represents a measurement from one of the measurement stations DMI operates in Denmark, Greenland and the Faroe Islands. The stations measure several different parameters including wind speed, precipitation, humidity, etc., and each row is a measurement of one parameter. The columns are:

1. `coordsx` First coordinate of measurement station.
2. `coordsy` Second coordinate of measurement station.
3. `created` Time measurement was entered in data base.
4. `observed` Time measurement was observed.
5. `parameterId` Measured parameter. See also the official documentation.
6. `stationId` ID of measurement station.
7. `value` Measured value.

1. Storage and Reading Files with Pandas

The goal of this exercise is to compare different method of loading and representing dataframes with pure Pandas.

1. The CSV file is stored as a zip file. Compare the time it takes for the following two approaches:

   a) First unpacking the zip file with the `unzip` command and then reading the CSV file with `read_csv`.

   b) Read the zip directly with `read_csv`.

   Which one is faster?

Answer: The exact times will depend on the hardware. If you are running on an interactive node, activities from other users may also have an effect. For me, unpacking the zip file took 4.7 s and reading the resulting CSV file with Pandas took 12.7 s. Reading the zippdd CSV file directly with

Pandas took 11.1 s. *Reading directly from the zip is faster.* This also has the benefit that we use less disk space, since we do not have to unpack the zip.

---

2. **Autolab** Load the data to a pandas dataframe (use `pd.read_csv()`). How much memory does the data frame occupy? Create a function `df_memsize` that takes a Pandas DataFrame as input and returns its size in bytes.

Answer: The dataframe uses around 2045 MB of memory, i.e., about 2 GB. Autolab exercise - Function omitted.

---

3. Check the columns and list all the changes you can do to save memory. Hint: check sections 7.1.2 and 7.1.3 on Fast Python for inspiration. You can use the following function summarize the dataframe:

```
def summarize_columns(df):
print(pd.DataFrame([
    (
        c,
        df[c].dtype,
        len(df[c].unique()),
        df[c].memory_usage(deep=True) // (1024**2)
    ) for c in df.columns
], columns=['name', 'dtype', 'unique', 'size (MB)']))
print('Total size:', df.memory_usage(deep=True).sum() / 1024**2, '
    MB')
```

Answer: The initial values are:

```
>>> summarize_columns(df)
         name    dtype   unique  size (MB)
0      coordsx  float64      224         62
1      coordsy  float64      219         62
2      created   object  8142495        652
3     observed   object    44640        597
4  parameterId   object       47        546
5    stationId    int64      247         62
6        value  float64    11532         62
Total size: 2045.0958003997803 MB
```

First, we can encode the 'created' and 'observed' columns as datetime values, similar to the book.

Next, we encode the 'parameterId' column. This is a string containing what parameter was measures. Since there is only a small number of possible parameters types, we can encode it as a categorial variable. This stores the values as indices into a dictionary holding the full values.

Next, since 'stationId' only contains 247 values, we could encode it as a smaller integer. The minimum 'stationId' value is 4203 and the maximum value is 34339. Therefore, despite there only being 247 values, we cannot encode as an 8-bit integer. Instead, we re-encode it as a 16-bit integer

Next, we want to see if can get away with a smaller float size for the 'value' column

```
>>> (df["value"] - df["value"].astype('float16')).abs().max()
inf
>>> (df["value"] - df["value"].astype('float32')).abs().max()
5.859375005456968e-05
```

While 16 bits will not work (due to overflow erros), we can get away with 32-bits

Finally, for the 'coordsx' and 'coordsy' columns, we can convert to a categorial variable. The reason this makes sense is that these columns store the coordinates of the measurement stations. As there are only a small number of stations compared to the number of data points, we can get away with storing the values in a dictionary.

Note that we *only* prioritized memory usage for these conversions. If you are doing computations with the columns later, you may want to make other conversions in order to trade off some memory for computation speed. As always, measure!

---

4. **Autolab** Perform the changes to reduce the memory usage. What is the new size of the dataframe? Create a function `reduce_dmi_df` that takes a Pandas DataFrame as input and returns a transformed DataFrame where at least 3 columns have had their memory use reduced.

Answer: Autolab exercise - answer omitted

---

2. Reading Files with Arrow

The goal of this exercise is to compare the time and memory size we get when using PyArrow to read the files instead of pandas. Check section 7.4 on Fast Python for help on this exercise.

1. **Autolab** Load the DMI data from January 2023, as we did in the last exercise, but this time using PyArrow. Compute the time it took to read the CSV, what was the speed up when comparing

with pandas? Create a function `pyarrow_load` that takes a path to a CSV file as input, loads the file with PyArrow and returns the PyArrow table.

Answer: Autolab exercise - Function omitted.

Reading the (unzipped) CSV file took me 3 seconds. Reading the CSV file pandas took 11.1 seconds. PyArrow is around 3.7 times faster.

---

2. **Autolab** Now convert the PyArrow table to a pandas data frame, how long does it take? Is the total time (time to load the data with PyArrow + the time to convert it to a pandas data frame) faster or slower than pure Pandas? Modify your previous function to return a Pandas DataFrame instead of a PyArrow table.

Answer: Autolab exercise - Function omitted.

Adding the Pandas conversion added about 700 ms for me. It is still faster to load the CSV file with PyArrow and then converting to Pandas than just loading with Pandas.

---

3. What was the size of the PyArrow table you got after loading the data? What was the size of the data frame after the conversion? Why are they different?

Answer: The size of the PyArrow table is 507 MB. After converting it back to a Pandas DataFrame, the size is 919 MB. Notice that several conversions have already been done on the Pandas DataFrame conpared to just loading it with Pandas: The 'created' and 'observed' columns have been converted to datetime types and the ‘stationId' has been converted to 64-bit integers.

The reason the Pandas DataFrame is bigger is mainly because the 'parameterId' is stored less efficiently. In Pandas it is an array of Python objects. In PyArrow all the strings are stored, one after the other, in one large buffer array and column then stores the index and length of each string in the buffer. This saves a lot of overhead compared to Python objects, which is why the PyArrow table is smaller.

---

4. Modify your loading code in order to perform (as many as possible of) the memory reducing operations you did in exercise 1.4. What is the new size of the PyArrow table?

Answer: We can perform conversion on load with `csv.ConvertOptions`:

```
convert_options = csv.ConvertOptions(
    column_types={
        'value': pa.float32(),
        'parameterId': pa.dictionary(pa.int32(), pa.string()),
        'coordsx': pa.dictionary(pa.int32(), pa.float64()),
        'coordsy': pa.dictionary(pa.int32(), pa.float64()),
    }
)
```

To perform the conversions, we pass `convert_options` to `csv.read_csv` - see the official documentation.

This reduces the size of the loaded PyArrow table to 312 MB.

---

3. Parquet Files

Parquet files offer a highly efficient representation for columnar data, optimizing storage, and query performance in data processing workflows. The data is stored in a binary file allowing to represent numbers in a much more compactly form than in text, as it is done for csv files. In this exercise we will explore the advantages of using Parquet files to store data. Check section 8.2 in Fast Python for help on this exercise.

1. **Autolab** Make a Python program that receives the path to a CSV file as a command line argument, loads the CSV file and finally saves it again as a Parquet file. What is the difference in size between the original CSV file and the Parquet file?

Answer: Autolab exercise - Program omitted.

The resulting parquet file is 86 MB.

---

2. Measure the times to read and write the DMI data to a parquet files and compare it to reading and writing CSV.

Answer: Writing the parquet file takes 1.6 seconds for me. Reading it with Pandas' `read_parquet` takes 1 second. Reading it with PyArrow's `read_table` takes 400 ms. Both are significantly faster than reading CSV files!

---

4. Pandas - Fast Operations

Our goal with this exercise is to compute the total amount of precipitation (i.e., rain) across all measurement stations. We will do this by summing all records with `parameterId` equal to `precip_past10min`. These hold the total precipitation in kg/m^2 or mm for this station over the past 10 minutes. Check sections 7.2.2 and 7.3.1 on Fast Python for help on this exercise.

A raw python implementation (i.e., not using pandas- or NumPy-based approaches) for could be:

```
def total_precip(df):
    total = 0.0
    for i in range(len(df)):
        row = df.iloc[i]
        if row['parameterId'] == 'precip_past10min':
            total += row['value']
    return total
```

1. Run the raw python implementation and time it. *Note:* Use the Pandas DataFrame method `sample` to subsample a smaller DataFrame, to avoid exessive wait times. How long did it take to execute?

   Answer: Your exact times will depend on the hardware. For me, sampling 100K rows (about 1.2\% of the data) takes about 5.5 seconds to process.

2. Build a pandas-based implementation using the `apply()` method to perform the same operation and time it. What was the speed up?

   Answer: We can implement the computation using `apply()` like this:

   ```
   def total_precip(df):
       total = df.apply(
           lambda row: (
               row['value'] if row['parameterId'] == '
                  precip_past10min' else 0.0
           ),
           axis=1
       ).sum()
       return total
   ```

   With this version, we can process the 100K subset in 780 ms, which is roughly 7 times faster. We can process the full dataset in 46 seconds. We get a value of 12548.63 as the total precipitation.

3. Now write a pure Pandas version using a vectorized approach. How long did it take to execute? What was the speed up in comparison with the two previous implementations?

Answer: A fully vectorized version may look like this:

```
1  def total_precip(df):
2      total = df[df['parameterId'] == 'precip_past10min']['value'].
           sum()
3      return total
```

For the 100K subset it takes 270 ms to process, which is 21 times faster than the original version and 3 times faster than using `apply`. For the full dataset, it takes 400 ms which is 112 times as fast as using `apply`. We again get a value of 12548.63 as the total precipitation.

---

4. **Autolab** Turn your previous implementation into a Python program that receives the path to a CSV file as a command line argument, loads the CSV, computes the total precipitation and finally prints the result (and only the result).

Answer: Autolab exercise - answer omitted

---

5. A significant portion of the work in the vectorized version is doing the indexing to extract the relevant rows. Rewrite the vectorized version to first use the `parameterId` column as an index. Excluding the time it takes to build the index, how long does it take to compute the total precipitation now? Including the time to build the index? When would one prefer one to the other? Hint: see section 7.2.1 in Fast Python.

Answer: Changing the vectorized version to use an index can be done like this:

```
1  df_pid = df.set_index('parameterId').sort_index()
2  precip = df_pid.loc['precip_past10min']['value'].sum()
```

Excluding the time to build the index, computing the total precipitation for the full dataset can now be done in 3 ms, which is 130 times faster than without an index and 1,500 times faster than using `apply`. *Note:* To time this, we must repeat the calculation several times to get a good time estimate. Including the time for the index, the compute time is 6.6 seconds, which is much slower.

Therefore, if are only doing a one-off computation it is not worth it to build the index *in this case*. However, if we were computing many queries for several different parameters, building the index could easily be worth it.